Programming Language Design

Module 8: Records

This is an open educational resource.
Feel free to submit fixes, improvements, and new material here.

Last time

We learned about parsing. Specifically:

- BNF and EBNF
- How recursive descent works
- Info on bottom-up parsing
- Info on how Haskell does it (precedence parsing)

The grammar is a critical part of a programming language's design.

This time

This is a short lecture about records.

A record in Haskell is basically a struct.

But the fact that it's taken us this long to talk about them should indicate that there are some wrinkles.

First, let's talk about structs in C. How do they work?

Structs

This is a struct in C:

```
typedef struct {
    char* name;
    int health;
    int mana;
} Character;
```

It has a few wrinkles:

- We typically start with `typedef` so we don't have to write `struct Character` whenever we want to declare one of them.
- In C, it's not clear who owns `name`. Who is responsible for freeing it? Is it `malloc`'d memory? Static memory? Stack memory?

Why structs?

[What is the point of structs?]

Struct benefits

Cache benefits: we put data that is used together physically next to each other. This has cache benefits sometimes (and also drawbacks sometimes--you have to decide whether it's worth it).

Design benefits: it's nice to put all the things a character needs in one place. Now we can pass a pointer to a character instead of 3 separate values.

Struct benefits (2)

The cache benefit is real, but pretty low level. Lower-level than we would typically think about in a functional programming language like Haskell.

The other benefit *is* something we care about. If there's a bunch of data that needs to go together, it's nice to group it up. This doesn't change just because we're using a functional language.

So let's see how to do it...

Basic data types

If you recall, a basic datatype declaration looks like this:

```
data Character = Character String Int Int
```

And there's nothing wrong with that, except for a small but important thing:
what do the fields mean?

There's a string, an int, and another int, but what are they?

Sure, we could document it, but it's much easier if the name *is* the documentation.
Software design is a lot easier if peoples' IDE naturally helps them understand what things mean instead of them having to consult a web page somewhere.

Record

Here's how it looks as a record:

```
data Character = Character {
    name :: String,
    health :: Int,
    mana :: Int
}
```

Looks straightforward enough, and works a lot of the way you would expect it.

First, we can now pass a character to a function:

```
isCharacterKnockedOut :: Character -> Bool
```

But how do we create a character?

Record constructors

It's basically the same as defining them, only we use `=` instead of `::` and values instead of types:

```
bob :: Character
bob = Character {
    name = "Bob",
    health = 100,
    mana = 20
}
```

What happens if we forget a field? Very important: we get an error. Haskell does not permit any constructor to not be fully defined: record or otherwise.

Questions?

Records are for constructors

One important thing to understand: we don't have to decide whether a type will be a record type or not.

Records are something that are associated with *constructors* rather than types.

It's possible for types to have record constructors and ordinary data constructors at the same time.

Here's an example...

Mixed record and data type constructors

```
data Character =
    Hero {
        name :: String,
        health :: Int,
        mana :: Int
    } |
    Monster Int -- maybe Monsters only have health
```

Now we have two ways to create a `Character`:

```
aHero :: Character
aHero = Hero { name = "Bob", health = 100, mana = 20 }

aMonster :: Character
aMonster = Monster 20
```

Important terms: data type vs constructor.

It's now very important to understand the difference between a constructor and a type.

A data type describes a set of values.

A constructor is like a function that is permitted to create those values.

One data type can have as many constructors as it wants.

These constructors can either be record constructors (with curly braces) or regular constructors (no braces).

Syntactic sugar

Deep down, records are product types, just like tuples and ordinary data types.

[Can anyone remind me what a product type is?]

[What about a sum type?]

Syntactic sugar (2)

In fact, behind the scenes, record constructors aren't really any different from non-record constructors.

We can see that if we examine the type of a constructor like this one in GHCI:

```
data Character = Hero { name :: String, health :: Int, mana :: Int }
ghci> :t Hero
Hero :: String -> Int -> Int -> Character
```

This type indicates we can actually call `Hero` as a function, and indeed, we still can, even though it's a record constructor:

```
bob = Hero "Bob" 100 20 -- works fine
bob = Hero { name = "Bob", health = 100, mana = 20 } -- also works
```

So why bother?

If record constructors are just data constructors behind the scenes, why even have them?

Well, look at this constructor call again:

```
Hero "Bob" 100 20
```

What is `100`? What is `20`? These end up being magic numbers.

We have to remember that health comes before mana. What if it were backwards?

When we have a lot of data we need to keep grouped together, it makes sense to use a record constructor, so that we can easily see what each data field means.

There's another benefit when it comes to taking data out we'll see soon...

Questions?

Getting data out of records

So, we know how to call a record constructor and put all our data in it.

How do we get the data out?

Well...[what do you think?]

Pattern matching records

Yup, pattern matching.

Here's how it works:

```
isKnockedOut :: Character -> Bool
isKnockedOut Character { health = h } = h <= 0
```

Here, `h` is a variable that matches whatever health is. We can make it more specific if we want:

```
healthIsExactlyZero :: Character -> Bool
healthIsExactlyZero Character { health = 0 } = True
healthIsExactlyZero _ = False
```

Is that all?

In C, we can do this:

```
Hero bob = { "Bob", 100, 25 };
puts(bob.name);
```

That is, we can use *dot notation* to get values out of the struct instance.

Can't we just do that in Haskell?

Well, sort of, but it's weird...

Record selectors in Haskell

Imagine we define these constructors:

```
data Character =
    Hero {
        name :: String,
        health :: Int,
        mana :: Int
    } |
    Monster {
        health :: Int,
        mana :: Int
    }
```

Haskell will define some functions that we can use to get this data back out without having to use pattern matching.

Record selectors (2)

These functions get automatically defined:

```
name :: Character -> String
health :: Character -> Int
mana :: Character -> Int
```

So we don't need pattern matching anymore:

```
bob = Hero { name = "Bob", health = 100, mana = 20 }
print $ health bob -- prints '100'
```

Great, seems reasonable. But there are some problems with doing it this way...

Record selector drawbacks

One major drawback is kind of subtle. Do you see it? What is the type of `name` again?

```
data Character =
    Hero {
        name :: String,
        health :: Int,
        mana :: Int
    } |
    Monster {
        health :: Int,
        mana :: Int
    }
```

Record selector drawbacks (2)

```
name :: Character -> String
```

That means we can pass *any* character to the `name` function and it will return its name.

But...what about `Monster`? Characters that use that constructor don't have a name...

You might think that there's some trick here, but no, Haskell actually just crashes:

```
ooze = Monster 20 0
print $ name ooze -- this just crashes
```

Questions?

Really?

This is honestly kind of a shocking hole in Haskell's type system. Normally Haskell is very strictly typed.

It's not as surprising when you consider that Haskell allows partial definitions of functions (definitions that don't cover all the values that could be passed).

However, it's kind of a questionable programming language design feature.

What programming language design principle is being violated?

Orthogonality

This is an area in which Haskell demonstrates poor orthogonality.

Orthogonality is a property of a programming language's features. If features are orthogonal, they can work together without interfering.

Here, field selectors are not a very orthogonal feature. It interferes with the global function definitions *and* the type system in a pretty annoying way.

Why?

Consider this situation?

```
data Blip = Blip { foo :: Int }
data Blop = Blop { foo :: Int }
```

Looks reasonable enough. We have two record constructors that have a field with the same name.

This is perfectly reasonable in C. What about Haskell?

Nope

You can't do this. You will get an error.

The first data definition is okay:

```
data Blip = Blip { foo :: Int }
```

This defines the `Blip` type and constructor and the field selector `foo :: Blip -> Int`.

However, the second one tries to define another field selector `foo :: Blop -> Int`
This is a multiple definition, which is not allowed.

Overloading?

Haskell does not allow trivial type overloading, and this is a pretty common restriction in functional languages.

In Java or C++, we can define a method that has the same name but takes different types. This is called *overloading*.

The problem with overloading is that it makes it so that the type of an expression involving a function can no longer be inferred. If there are two versions of `foo`, what is the type of `show . foo`?

It could either be `Blip -> String` or `Blop -> String`. There is no way to know.

Overloading (2)

Rather than say "sometimes type inference will fail", Haskell instead says "we can always get the type of simple expressions. It's on you to make sure the names don't conflict."

This isn't as weird as it sounds. I've seen functional programmers say "if two functions have different types then they should have different names. Don't overload by type."

Unfortunately, it interacts poorly with the field selector feature in Haskell.

Actually fixing it

One thing we can do is put our types with record constructors inside of modules:

```
module Character where
    data Character =
        Hero { health :: Int } |
        Monster { health :: Int } |
        Npc { health :: Int }
```

Now, someone can `import qualified Character` in another file.

The `qualified` specifier means they have to write `Character.` in front of anything in the module to refer to it.

So now they write `Character.health` instead of just `health`, preventing conflicts with other record constructors.

Is it fixed?

Unfortunately, this solution has some drawbacks.

First, modules have to go in their own files. This is really annoying, and it scatters data type definitions around our codebase.

Second, it means that the type itself needs to be referred to as `Character.Character`.

There are various ways of importing with different tradeoffs that can work around this annoyance, but none really fixes every problem satisfactorily in my opinion.

Personally, modules are another weak point in Haskell's feature set for me.

Personal opinion time

I've been pretty Haskell-positive so far, and I still am: it's a very elegant language.

But sometimes the designers take elegance too far. If I had to propose a simple change to improve Haskell I would change how field selectors work.

Specifically, I would require the name of the type when using them: `print $ Character.health bob` instead of just `print $ health bob`.

This is already what happens if you use modules, but I would make it the default. Now we can make as many record constructors with `health` fields as we want.

Personal opinion time (2)

My proposal isn't perfect. Modules are already a thing.

Instead, I could make `Character` by itself still refer to a type, and I would create an anonymous module that stores the field selectors.

Is that a bad solution? It involves some hidden behavior. It might be *surprising*, which is a property we want to avoid when designing programming languages.

Maybe all types should be modules. The module could store meta information about the type. But that would make the language more complicated. A type would now be a kind of module, instead of a totally orthogonal concept.

Personal opinion time (3) -- language design is hard

These kinds of tradeoffs are why there is no perfect programming language.

My proposal also *doesn't* fix the issue with calling selectors on constructors that don't have that field. To fix that, we'd need record constructors to be unique within a type. That's a pretty major limitation, although it might be worth it for better type safety. [Thoughts?]

Record data extraction summary

In summary, we have two features for getting data out of a record:

1. Pattern matching: a pattern specifies the fields we care about and what values they have. If the value is a variable, it will match anything, just like we're used to: `someFunc Constructor {foo = 100, bar = b} = b` will return the `bar` field, but only if the `foo` field is exactly 100.
2. Data selectors: Haskell defines a function for every unique field of a record constructor. We can call these functions to get data out: `print $ health bob` will print `100` if `bob = Hero { name = "Bob", health = 100, mana = 25 }`

Data selectors have limitations: every field name must be unique within each type. Different constructors of a type can use the same name, but not different constructors of different types. They can crash if the data is absent for a given constructor.

Questions?

Record practice

1. Create a data type called `PongActor` with two record constructors: `Paddle` and `Ball`
2. Make both constructors have x,y coordinates and width and height. Store them however you want.
3. Give ball fields for x and y velocity as well.